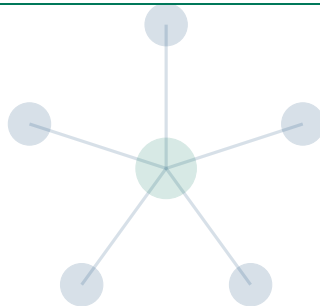


Linux for Computational Chemistry

Lecture 4: Remote Computing and HPC Job Scheduling

Emilio Martínez Núñez

Master in Theoretical and Computational Chemistry



Cluster architecture

Remote access

Archiving and Compression

Environment modules

The scheduler: SLURM

Array jobs

Cluster architecture

Anatomy of an HPC cluster

- **Login nodes:** where you land when you connect — for editing, compiling, submitting jobs. *Never* run heavy calculations here.
- **Compute nodes:** hundreds of machines that actually run your jobs, allocated by a scheduler
- **Shared filesystem:** your `/home` and `/scratch` are visible from every node
- You do not choose a compute node directly — you *request* resources and the scheduler assigns one

Remote access

Connecting: ssh

```
ssh jperez@cluster.usc.es  
ssh -p 2222 jperez@cluster.usc.es # non-default port
```

```
ssh-keygen -t ed25519 # generate a keypair (once)  
ssh-copy-id jperez@cluster.usc.es # install your public key remotely
```

Note

Key-based login means no password prompt and is required by most clusters. The private key never leaves your laptop.

Moving files: scp and rsync

```
scp opt.com jperez@cluster.usc.es:/scratch/jperez/butanol/  
scp jperez@cluster.usc.es:/scratch/jperez/butanol/opt.log .  
  
rsync -avz local_dir/ jperez@cluster.usc.es:/scratch/jperez/project/  
rsync -avz --progress jperez@cluster.usc.es:/scratch/jperez/project/ local_dir/
```

Tip

Prefer `rsync` over `scp` for anything beyond a single file: it only transfers what changed, and `-a` preserves permissions and timestamps.

Exercise 4.1 — Connecting and transferring

Exercise 4.1

Using the training account provided:

1. Connect via `ssh` to the training cluster.
2. From your laptop, copy a local input file to your `/scratch` space with `scp`.
3. Use `rsync -avz` to synchronize an entire local project folder to the cluster.
4. Modify one file locally and re-run the same `rsync` command — confirm only that one file is transferred.

Archiving and Compression

tar: packing a whole directory into one file

```
tar -czvf butanol_results.tar.gz butanol_project/ # Create, gZip, Verbose, File
tar -xzvf butanol_results.tar.gz # eXtract
tar -tzvf butanol_results.tar.gz # list contents, no extraction
```

- c=create, x=extract, t=list, z=gzip compression, v=verbose, f=filename follows
- A .tar.gz (or .tgz) is the standard way to move a whole project as one file

Individual file compression

```
gzip opt.log # opt.log -> opt.log.gz (original removed)  
gunzip opt.log.gz # reverse it  
bzip2 opt.log # slower, usually smaller than gzip  
zip results.zip *.log # cross-platform, handy for sending to Windows/macOS users
```

Tip

Compress before transferring over scp/rsync on a slow connection — a tar.gz of many small log files both transfers faster and avoids the per-file overhead of copying thousands of files individually.

Exercise 4.0 — Archive before you transfer

Exercise 4.0

1. Package your entire `2-butanol_project` directory into a single `project.tar.gz` (exclude the `.git` folder with tar's `-exclude=.git` option).
2. Check its contents with `tar -tzvf` *without* extracting.
3. Transfer it to the cluster with `scp` or `rsync`.
4. On the cluster, extract it and confirm the directory structure is intact.

Solution

```
tar --exclude=.git -czvf project.tar.gz 2-butanol_project/  
tar -tzvf project.tar.gz  
scp project.tar.gz jperez@cluster.usc.es:/scratch/jperez/  
ssh jperez@cluster.usc.es "cd /scratch/jperez && tar -xzf project.tar.gz"
```

Environment modules

Environment modules

```
module avail # list all installable software
module avail gaussian # search for a specific package
module load gaussian/16 # load a specific version into your PATH
module list # see what is currently loaded
module purge # unload everything
module unload gaussian/16 # unload one module
```

Note

Clusters host many versions of the same software (different compilers, MPI libraries...). Modules avoid conflicts by keeping only what you explicitly load in your environment.

Exercise 4.2

1. Search the module system for any available quantum chemistry package (e.g. `gaussian`, `orca`, `nwchem`).
2. Load one version and confirm it appears in `module list`.
3. Check that the corresponding executable is now on your PATH (hint: `which g16` or similar).
4. Purge your modules and confirm the executable is no longer found.

The scheduler: SLURM

Why a scheduler?

- A cluster is shared by hundreds of users — you cannot just run your job whenever you like
- You submit a **job**: a description of what to run and what resources it needs
- The scheduler (SLURM here) queues it and starts it on compute nodes when resources are free
- Key resources to request: number of nodes/cores, memory, walltime, and a **partition** (queue)

Anatomy of an sbatch script

```
#!/bin/bash
#SBATCH --job-name=butanol_opt
#SBATCH --partition=standard
#SBATCH --nodes=1
#SBATCH --ntasks=8
#SBATCH --mem=16G
#SBATCH --time=04:00:00
#SBATCH --output=slurm-%j.out

module load gaussian/16
g16 < opt.com > opt.log
```

- #SBATCH lines are directives read by SLURM, not comments to bash
- %j in the output filename is replaced by the job ID

Submitting and monitoring jobs

```
sbatch run_opt.slurm # submit the job, returns a job ID  
squeue -u jperez # see your jobs in the queue  
squeue --me # shorthand on many clusters  
scancel 123456 # cancel a specific job  
sacct -j 123456 # accounting info after it finishes  
sinfo # see partitions and node availability
```

Exercise 4.3

Write `run_opt.slurm` that:

1. Requests 1 node, 4 tasks, 8 GB memory, and a 1-hour walltime, in the standard partition.
2. Loads the Gaussian module.
3. Runs `g16` on `opt.com`, redirecting output to `opt.log`.
4. Submit it with `sbatch` and confirm it appears in `squeue`.
5. Once finished, inspect `slurm-<jobid>.out` and `opt.log`.

Solution

```
#!/bin/bash
#SBATCH --job-name=opt_test
#SBATCH --partition=standard
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --mem=8G
#SBATCH --time=01:00:00
#SBATCH --output=slurm-%j.out
```

```
module load gaussian/16
g16 < opt.com > opt.log
```

```
sbatch run_opt.slurm
queue --me
```

Array jobs

Array jobs: many similar tasks at once

```
#!/bin/bash
#SBATCH --job-name=conf_scan
#SBATCH --array=1-10
#SBATCH --ntasks=4
#SBATCH --mem=8G
#SBATCH --time=02:00:00
#SBATCH --output=slurm-%A_%a.out

module load gaussian/16
g16 < conf${SLURM_ARRAY_TASK_ID}.com > conf${SLURM_ARRAY_TASK_ID}.log
```

- One sbatch submission creates 10 independent jobs
- \$SLURM_ARRAY_TASK_ID takes each value from 1 to 10, one per task
- Ideal for conformer scans, parameter sweeps, batches of single points

Exercise 4.4 — Array job for a conformer scan

Exercise 4.4

You have ten Gaussian input files `conf1.com` ... `conf10.com`.

1. Write a SLURM array script that runs all ten as an array job, each with 4 tasks and 8 GB memory.
2. Use `$SLURM_ARRAY_TASK_ID` to build the correct input/output filenames for each task.
3. Submit it, then use `squeue -me` to observe the individual array tasks.
4. Once complete, adapt your `check_jobs.sh` script from Lecture 3 to summarize the results.

Solution 4.4

Solution

```
#!/bin/bash
#SBATCH --job-name=conf_scan
#SBATCH --array=1-10
#SBATCH --ntasks=4
#SBATCH --mem=8G
#SBATCH --time=02:00:00
#SBATCH --output=slurm-%A_%a.out

module load gaussian/16
g16 < conf${SLURM_ARRAY_TASK_ID}.com > conf${SLURM_ARRAY_TASK_ID}.log
```

```
sbatch conf_scan.slurm
queue --me
# after completion:
bash check_jobs.sh
```

- `ssh/scp/rsync` get you and your files onto the cluster
- `module` manages which software is on your `PATH`
- `sbatch/squeue/scancel` submit and manage jobs under SLURM
- Array jobs turn "run this 100 times" into a single submission

Exercise 4.5 (Homework)

1. Extend your array job script so that, after all array tasks would normally finish, a final dependent job (`sbatch -dependency=afterany:<jobid>`) automatically runs `check_jobs.sh` and emails or prints the summary.
2. Research briefly, and be ready to explain in one sentence, the difference between `-dependency=afterok` and `-dependency=afterany`.